

# Python for Finance - M1 EF Dauphine

Matéo Molinaro

September 2026

## 1 What Is a High-Level Programming Language?

Python is considered a **high-level programming language**. This means that it provides a high level of abstraction from the physical operation of the computer.

In other words, Python allows the programmer to focus primarily on **what problem should be solved**, rather than on the low-level details of **how the computer executes the corresponding operations**.

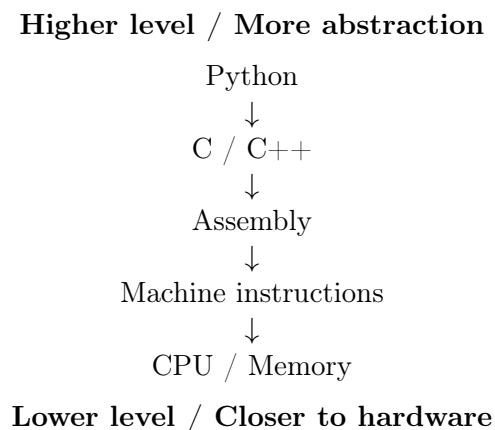
For example, consider the following Python code:

```
returns = prices.pct_change()
mean_return = returns.mean()
```

The programmer directly expresses the desired operations: compute returns and then calculate their average.

At a lower level, these operations ultimately involve many additional details, such as memory allocation, numerical representations, CPU instructions, registers, and memory access.

Programming languages can therefore be viewed as different levels of abstraction from the underlying hardware:



For example, in Python:

```
x = 42
```

Python automatically handles many operations related to the creation and management of the object representing the value 42. These include, among others, memory allocation, type management, and memory deallocation.

Lower-level languages such as C provide the programmer with substantially more explicit control over these mechanisms.

## Key Idea

A **high-level programming language** abstracts away many of the low-level details of computer hardware, allowing programmers to express the logic of the problem more directly.

Importantly, **high-level does not mean more advanced or more difficult**. It refers to the **level of abstraction from the hardware**.

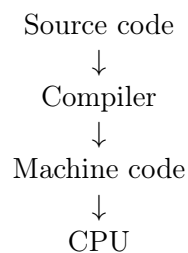
## 2 Compiled vs. Interpreted Languages

Another important distinction concerns how source code is transformed into instructions that can ultimately be executed by the computer.

### 2.1 Compiled Languages

In a traditionally compiled language such as C or C++, the source code is processed by a **compiler** before execution.

Conceptually:



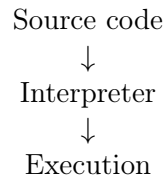
For example, a C program is generally compiled into machine instructions before it is executed. The compiler can analyze and optimize the program in advance. The resulting machine code can then be executed directly by the processor.

This is one reason why compiled languages such as C and C++ generally provide very high execution performance.

### 2.2 Interpreted Languages

In the traditional model of an interpreted language, the source code is executed through an **interpreter** at runtime.

Conceptually:



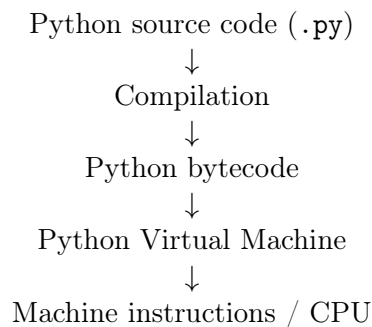
Python is therefore commonly described as an **interpreted language**.

However, this description is a simplification.

### 2.3 What Actually Happens with Python?

The standard implementation of Python, called **CPython**, first compiles Python source code into an intermediate representation called **bytecode**.

The process can be represented as:



The bytecode is then executed by the Python interpreter, more precisely by the Python Virtual Machine.

Python can therefore be viewed as involving **both compilation and interpretation**:

Python source code is typically **compiled into bytecode**, which is then **executed by the Python interpreter**.

## 3 Programming Paradigms

A **programming paradigm** is a general approach to structuring and expressing the logic of a computer program.

Python is a **multi-paradigm programming language**, meaning that it supports several programming styles. In particular, Python supports three major paradigms:

- **Imperative programming**
- **Object-Oriented Programming (OOP)**
- **Functional programming**

These paradigms are not mutually exclusive. In practice, Python programs often combine elements from all three.

### 3.1 Imperative Programming

In the **imperative paradigm**, a program is expressed as a sequence of instructions that modify the **state** of the program.

The programmer explicitly specifies **how** a computation should be performed, step by step.

For example:

```
prices = [100, 105, 103, 110]

returns = []

for i in range(1, len(prices)):
    r = prices[i] / prices[i - 1] - 1
    returns.append(r)
```

The program explicitly describes the sequence of operations:

Initialize variables → Iterate over prices → Compute return → Modify **returns**

The state of the variable **returns** changes progressively during execution.

Typical features of imperative programming include:

- variable assignments;
- loops such as **for** and **while**;
- conditional statements such as **if**;
- explicit modification of program state;
- execution of instructions in a specified order.

#### Key idea.

Imperative programming describes **how to perform a computation** through a sequence of instructions that modify the program state.

Procedural programming, where instructions are organized into reusable procedures or functions, can be viewed as a form of imperative programming.

### 3.2 Object-Oriented Programming

**Object-Oriented Programming (OOP)** organizes a program around **objects**.

An object combines:

- **data**, called **attributes**;
- **behavior**, represented by **methods**.

Objects are typically created from **classes**, which define their structure and behavior.

For example, a simple portfolio can be represented as:

```
class Portfolio:

    def __init__(self, weights):
        self.weights = weights

    def compute_return(self, asset_returns):
        return sum(
            w * r
            for w, r in zip(self.weights, asset_returns)
        )

portfolio = Portfolio([0.5, 0.3, 0.2])

r = portfolio.compute_return([0.02, -0.01, 0.03])
```

Here, the object `portfolio` contains both:

| State                | Behavior                      |
|----------------------|-------------------------------|
| <code>weights</code> | <code>compute_return()</code> |

This approach is particularly useful for organizing large applications.

For example, a quantitative backtesting system could contain:

```
DataHandler
SignalModel
Portfolio
RiskModel
Backtester
PerformanceAnalyzer
```

Each class is responsible for a specific part of the system.

### Key idea.

Object-oriented programming organizes programs around **objects that combine data and behavior**.

## 3.3 Functional Programming

The **functional paradigm** treats computation primarily as the evaluation and composition of **functions**.

A central idea is to avoid unnecessary changes in state and favor functions whose output depends only on their inputs.

For example:

```
def compute_return(p0, p1):  
    return p1 / p0 - 1
```

The function takes inputs and produces an output without modifying variables outside the function.

Conceptually:

Inputs  $\rightarrow$  Function  $\rightarrow$  Output

For the same inputs, a **pure function** always produces the same output and does not create side effects.

Python also provides several tools inspired by functional programming, including:

- `lambda` expressions;
- `map()`;
- `filter()`;
- `reduce()`;
- comprehensions;
- higher-order functions.

For example:

```
returns = [0.02, -0.01, 0.03, -0.02]  
  
positive_returns = list(  
    filter(lambda r: r > 0, returns)  
)
```

Functional programming is particularly useful when transformations can be expressed as compositions of independent functions.

### Key idea.

Functional programming describes computations primarily through the **evaluation and composition of functions**, while limiting changes to program state and side effects.

## 3.4 Comparing the Three Paradigms

The same problem can often be approached using different programming paradigms.

| Paradigm        | Central Idea                                                                        | Typical Python Features                                                                  |
|-----------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Imperative      | Execute instructions that progressively modify program state.                       | Assignments, <code>for</code> , <code>while</code> , <code>if</code>                     |
| Object-Oriented | Organize data and behavior into interacting objects.                                | Classes, objects, attributes, methods                                                    |
| Functional      | Express computations as functions and transformations while limiting state changes. | Functions, <code>lambda</code> , <code>map</code> , <code>filter</code> , comprehensions |

A useful simplified distinction is:

|                        |                                                      |
|------------------------|------------------------------------------------------|
| <b>Imperative</b>      | “Which instructions should the computer execute?”    |
| <b>Object-Oriented</b> | “Which objects should represent the system?”         |
| <b>Functional</b>      | “Which functions transform the inputs into outputs?” |

Python allows these approaches to be combined. For example, a quantitative finance application may use **objects** to represent portfolios and strategies, **imperative code** to control a backtest through time, and **functional transformations** to compute signals and process data.

Therefore, Python is not only a **high-level** and commonly described as an **interpreted** language, but also a **multi-paradigm programming language**.

## 4 Sets

A **set** is a built-in Python data structure that stores a collection of **unique elements**.

Sets are particularly useful when:

- duplicate values should be removed;
- we want to test whether an element belongs to a collection;
- we want to perform mathematical set operations such as unions, intersections, and differences.

### 4.1 Creating a Set

A set can be created using curly brackets:

```
stocks = {"AAPL", "MSFT", "GOOG"}
```

Unlike a list, a set contains only **unique values**.

For example:

```
stocks = {"AAPL", "MSFT", "AAPL", "GOOG"}

print(stocks)
```

produces a set containing only:

```
{"AAPL", "MSFT", "GOOG"}
```

The duplicate "AAPL" is automatically removed.

A list can also be converted into a set:

```
stocks = ["AAPL", "MSFT", "AAPL", "GOOG"]  
  
unique_stocks = set(stocks)
```

This is a common way to remove duplicates from a collection.

## 4.2 Set vs. List

Sets and lists represent collections of elements, but they have important differences.

| Property           | List             | Set                            |
|--------------------|------------------|--------------------------------|
| Syntax             | [a, b, c]        | {a, b, c}                      |
| Duplicates         | Allowed          | Not allowed                    |
| Ordered            | Yes              | No guaranteed positional order |
| Indexing           | Yes              | No                             |
| Membership testing | Generally $O(n)$ | Average $O(1)$                 |

For example, a list can be indexed:

```
stocks = ["AAPL", "MSFT", "GOOG"]  
  
print(stocks[0])
```

whereas a set cannot:

```
stocks = {"AAPL", "MSFT", "GOOG"}  
  
stocks[0]    # Error
```

A set should therefore be viewed primarily as a collection based on **membership**, rather than position.

## 4.3 Adding and Removing Elements

Elements can be added using `add()`:

```
stocks = {"AAPL", "MSFT"}
```

```
stocks.add("GOOG")
```

Elements can be removed using `remove()`:

```
stocks.remove("MSFT")
```

or `discard()`:

```
stocks.discard("MSFT")
```

The difference is that `remove()` raises an error if the element does not exist, whereas `discard()` does not.

## 4.4 Membership Testing

One of the main advantages of sets is efficient membership testing.

For example:

```
universe = {"AAPL", "MSFT", "GOOG", "NVDA"}
```

```
if "AAPL" in universe:  
    print("Stock belongs to the universe")
```

Membership testing in a Python set is, on average,

$$O(1),$$

because sets are implemented using **hash tables**.

By comparison, searching for an element in a list generally requires

$$O(n)$$

operations in the worst case.

## 4.5 Mathematical Set Operations

Python sets naturally support standard mathematical set operations.

Suppose we have two investment universes:

```
universe_A = {"AAPL", "MSFT", "GOOG", "NVDA"}  
universe_B = {"MSFT", "NVDA", "AMZN", "META"}
```

## Union

The **union** contains all elements appearing in either set:

$$A \cup B.$$

In Python:

```
universe_A | universe_B
```

which gives:

```
{"AAPL", "MSFT", "GOOG", "NVDA", "AMZN", "META"}
```

## Intersection

The **intersection** contains elements appearing in both sets:

$$A \cap B.$$

In Python:

```
universe_A & universe_B
```

which gives:

```
{"MSFT", "NVDA"}
```

## Difference

The **difference** contains elements belonging to the first set but not to the second:

$$A \setminus B.$$

In Python:

```
universe_A - universe_B
```

which gives:

```
{"AAPL", "GOOG"}
```

## Symmetric Difference

The **symmetric difference** contains elements appearing in exactly one of the two sets:

$$A \Delta B.$$

In Python:

```
universe_A ^ universe_B
```

## 4.6 An Important Detail: The Empty Set

An empty set must be created using:

```
empty_set = set()
```

and **not**:

```
empty_set = {}
```

Indeed, `{}` creates an empty **dictionary**, not an empty set.

## 4.7 Summary

A Python **set** is an unordered collection of unique elements, optimized for membership testing and mathematical set operations.

The most important operations to remember are:

| Operation            | Python                 | Mathematics     |
|----------------------|------------------------|-----------------|
| Membership           | <code>x in A</code>    | $x \in A$       |
| Union                | <code>A   B</code>     | $A \cup B$      |
| Intersection         | <code>A &amp; B</code> | $A \cap B$      |
| Difference           | <code>A - B</code>     | $A \setminus B$ |
| Symmetric difference | <code>A ^ B</code>     | $A \Delta B$    |

# 5 Hashability in Python

## 5.1 What Does “Hashable” Mean?

In Python, an object is said to be **hashable** if Python can associate it with a **hash value** that remains stable during the object’s lifetime.

A hash value can be obtained using the built-in function:

```
hash(object)
```

For example:

```
hash(42)
hash("Python")
hash((1, 2, 3))
```

Integers, strings, and tuples of hashable objects are generally hashable.

## 5.2 What Is a Hash Value?

A **hash function** transforms an object into an integer:

$$h : \text{Object} \longrightarrow \mathbb{Z}$$

Conceptually:

$$\text{"AAPL"} \xrightarrow{\text{hash function}} \text{integer hash value}$$

Python uses these hash values internally to efficiently locate objects in **hash tables**.

Hash tables are the data structure underlying:

- dictionaries (`dict`);
- sets (`set`).

This is why membership testing in a set or dictionary is very efficient on average:

$$O(1)$$

## 5.3 Why Is a List Not Hashable?

A Python `list` is **mutable**: its contents can be modified after the list has been created.

For example:

```
x = [1, 2, 3]

x.append(4)

print(x)
# [1, 2, 3, 4]
```

If lists were hashable, changing the contents of a list could change the information from which its hash would naturally be computed.

Conceptually:

$$[1, 2, 3] \xrightarrow{\text{hash}} h_1$$

but after modifying the list:

$$[1, 2, 3, 4] \xrightarrow{\text{hash}} h_2.$$

This would violate the requirement that the hash of an object used as a hash-table key must remain stable.

Python therefore deliberately makes lists **unhashable**.

For example:

```
x = [1, 2, 3]
```

```
hash(x)
```

raises:

```
TypeError: unhashable type: 'list'
```

## 5.4 Practical Consequences

Because lists are not hashable, they cannot be used as:

- elements of a `set`;
- keys of a `dict`.

For example, the following is invalid:

```
my_set = {[1, 2], [3, 4]}
```

Similarly:

```
my_dict = {  
    [1, 2]: "A"  
}
```

Both raise a `TypeError`.

## 5.5 List vs. Tuple

This explains an important difference between lists and tuples.

A list is mutable:

```
x = [1, 2, 3]  
x[0] = 10
```

and is therefore not hashable.

A tuple is immutable:

```
x = (1, 2, 3)
```

and, provided all its elements are themselves hashable, it can be hashable:

```
hash((1, 2, 3))
```

Therefore, a tuple can be used as a dictionary key:

```
prices = {
    ("AAPL", "2026-09-07"): 220.5,
    ("MSFT", "2026-09-07"): 410.2
}
```

## 5.6 Summary

| Type  | Mutable? | Hashable?    |
|-------|----------|--------------|
| int   | No       | Yes          |
| float | No       | Yes          |
| str   | No       | Yes          |
| tuple | No       | Usually yes* |
| list  | Yes      | No           |
| set   | Yes      | No           |

*\*A tuple is hashable only if all of its elements are hashable.*

**Key idea:** A hashable object has a stable hash value and can therefore be used in hash-based structures such as dictionary keys or set elements. A list is mutable, so Python deliberately makes it unhashable.

## 6 Correction: for and while Loops

### 6.1 for Loops

Consider the following list:

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
```

#### Loop 1

```
for stock in my_stocks:
    print(stock)
```

The `for` loop successively assigns each element of `my_stocks` to the variable `stock`.

The output is therefore:

```
AMZN
TSLA
AAPL
META
```

Conceptually:

$$\text{stock} = \text{AMZN} \rightarrow \text{TSLA} \rightarrow \text{AAPL} \rightarrow \text{META}.$$

## Loop 2

Consider:

```
for i in range(1, 10):
    print(i)
```

The function

`range(start, stop)`

generates integers from `start` up to, but **excluding**, `stop`.

Therefore,

$$\text{range}(1, 10) = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

The output is:

```
1
2
3
4
5
6
7
8
9
```

**Important:** the upper bound of `range()` is excluded.

## Loop 3: Iterating over a Dictionary

Consider:

```

my_prices = [133.62, 889.36, 167.23, 161.11]

my_ptf = dict(zip(my_stocks, my_prices))

for stock, price in my_ptf.items():
    print(f"Le prix de {stock} est {price}")

```

First, `zip()` associates the elements having the same positions:

```

AMZN  $\longleftrightarrow$  133.62,
TSLA  $\longleftrightarrow$  889.36,
AAPL  $\longleftrightarrow$  167.23,
META  $\longleftrightarrow$  161.11.

```

Then,

```
dict(zip(my_stocks, my_prices))
```

creates the dictionary:

```

{
    'AMZN': 133.62,
    'TSLA': 889.36,
    'AAPL': 167.23,
    'META': 161.11
}

```

The method `items()` provides the key-value pairs of the dictionary. At each iteration, Python performs tuple unpacking into:

```
stock, price
```

The output is:

```

Le prix de AMZN est 133.62
Le prix de TSLA est 889.36
Le prix de AAPL est 167.23
Le prix de META est 161.11

```

## 6.2 while Loops

Consider:

```

portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']

```

```
while True:
```

```

portfolio1.append(portfolio2[-1])
portfolio2.pop()

if not bool(portfolio2):
    break

print(portfolio1)

```

### Why while True?

A **while** loop continues executing as long as its condition evaluates to **True**.

For example:

```

while condition:
    ...

```

Using:

```

while True:

```

therefore creates an **infinite loop**, because the condition **True** can never become **False**.

Conceptually:

$$\text{True} \rightarrow \text{execute} \rightarrow \text{True} \rightarrow \text{execute} \rightarrow \dots$$

The loop must consequently be terminated explicitly using:

```

break

```

In this example, the stopping condition is:

```

if not bool(portfolio2):
    break

```

### Understanding the Stopping Condition

For a list, Python follows the convention:

```

bool(non-empty list) = True,
bool(empty list) = False.

```

Therefore, when:

```

portfolio2 = []

```

we have:

```
bool(portfolio2)
# False
```

and consequently:

```
not bool(portfolio2)
# True
```

The instruction `break` is then executed and the loop stops.

## Execution Step by Step

Initially:

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']
```

At each iteration, the last element of `portfolio2` is first copied into `portfolio1` and then removed from `portfolio2`.

| Iteration | Element transferred | portfolio2 after pop() |
|-----------|---------------------|------------------------|
| 1         | F                   | [V, TTE, FTKE]         |
| 2         | FTKE                | [V, TTE]               |
| 3         | TTE                 | [V]                    |
| 4         | V                   | []                     |

Notice that the elements are transferred in **reverse order**, because index `-1` and `pop()` operate on the last element.

The final output is:

```
['AMZN', 'TSLA', 'AAPL', 'META', 'F', 'FTKE', 'TTE', 'V']
```

## 6.3 A More Concise Variant

The previous loop can be written as:

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']
```

```
while True:
    portfolio1.append(portfolio2.pop())

    if not bool(portfolio2):
        break
```

```
print(portfolio1)
```

## Why Does This Work?

The important point is that `pop()` does **two things**:

1. it removes an element from the list;
2. it **returns the removed element**.

For example:

```
x = ['A', 'B', 'C']
```

```
value = x.pop()
```

After execution:

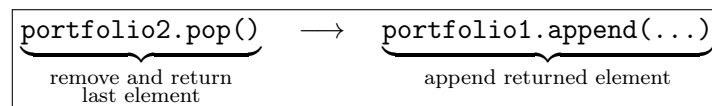
```
value  
# 'C'
```

```
x  
# ['A', 'B']
```

Therefore:

```
portfolio1.append(portfolio2.pop())
```

can be understood as:



Thus the following two instructions:

```
portfolio1.append(portfolio2[-1])  
portfolio2.pop()
```

can be replaced by the single instruction:

```
portfolio1.append(portfolio2.pop())
```

## Why Is It More Efficient / Parsimonious?

The first version performs two logically separate operations:

```
portfolio1.append(portfolio2[-1])  # access last element  
portfolio2.pop()                  # remove last element
```

The second version directly uses the value returned by `pop()`:

```
portfolio1.append(portfolio2.pop())
```

It is therefore:

- **shorter**: one instruction instead of two;
- **less redundant**: the last element does not need to be explicitly accessed before being removed;
- **clearer**: the code directly expresses the operation “move the last element from one list to the other”.

Both versions have the same overall time complexity:

$$O(n)$$

for transferring  $n$  elements, since `pop()` from the end of a Python list and `append()` are both  $O(1)$  amortized operations.

Thus, “more efficient” here should mainly be understood as **more concise and less redundant**, rather than as a change in asymptotic computational complexity.

### Additional Simplification

The condition:

```
if not bool(portfolio2):
```

can itself be simplified.

Python automatically interprets an empty list as **False** and a non-empty list as **True**. Therefore, the more idiomatic expression is:

```
if not portfolio2:
    break
```

The loop can therefore be written as:

```
while True:
    portfolio1.append(portfolio2.pop())

    if not portfolio2:
        break
```

An even simpler solution is to put the condition directly in the **while** statement:

```
while portfolio2:
    portfolio1.append(portfolio2.pop())
```

This means:

“Continue executing the loop while `portfolio2` is not empty.”

For this particular problem, this last version is the most concise and idiomatic Python implementation.

## 7 Correction: Conditional Statements

Consider the following code:

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
stocks2 = ['V', 'TTE', 'AAPL', 'MS', 'AMZN']

for stocks in stocks2:
    if stocks in my_stocks:
        print(f'{stocks} is all ready in the first portfolio')
    else:
        my_stocks.append(stocks)

print(my_stocks)
```

### 7.1 Understanding the Condition

The instruction

```
if stocks in my_stocks:
```

tests whether the current stock from `stocks2` is already present in `my_stocks`.

The operator `in` is a **membership operator**:

$$x \text{ in collection} = \begin{cases} \text{True,} & \text{if } x \text{ belongs to the collection,} \\ \text{False,} & \text{otherwise.} \end{cases}$$

If the stock already exists, a message is printed.

Otherwise, the `else` block is executed and the stock is added to `my_stocks` using:

```
my_stocks.append(stocks)
```

### 7.2 Step-by-Step Execution

Initially:

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
```

The loop then examines each element of `stocks2`.

| Stock | Already present? | Condition | Action        |
|-------|------------------|-----------|---------------|
| V     | No               | False     | Append V      |
| TTE   | No               | False     | Append TTE    |
| AAPL  | Yes              | True      | Print message |
| MS    | No               | False     | Append MS     |
| AMZN  | Yes              | True      | Print message |

Thus, the evolution of `my_stocks` is:

```
[AMZN, TSLA, AAPL, META]
↓ + V
[AMZN, TSLA, AAPL, META, V]
↓ + TTE
[AMZN, TSLA, AAPL, META, V, TTE]
↓ + MS
[AMZN, TSLA, AAPL, META, V, TTE, MS]
```

### 7.3 Output

The two stocks already present in the first portfolio generate a message:

```
AAPL is all ready in the first portfolio
AMZN is all ready in the first portfolio
```

Finally, the instruction

```
print(my_stocks)
```

produces:

```
['AMZN', 'TSLA', 'AAPL', 'META', 'V', 'TTE', 'MS']
```

Therefore, the complete output is:

```
AAPL is all ready in the first portfolio
AMZN is all ready in the first portfolio
['AMZN', 'TSLA', 'AAPL', 'META', 'V', 'TTE', 'MS']
```

### 7.4 Important Detail

The condition is evaluated against `my_stocks`, which is **modified during the execution of the loop**.

For example, after the first iteration:

```
my_stocks.append('V')
```

the list becomes:

```
['AMZN', 'TSLA', 'AAPL', 'META', 'V']
```

Consequently, subsequent membership tests use the **updated list**, not the original one.

**Key idea:** The `if` statement checks a condition. If the condition is `True`, the `if` block is executed; otherwise, the `else` block is executed.

## 8 Methods: Referencing vs. Calling a Method

Consider the following class:

```
class Voiture:

    def __init__(self, marque: str, couleur: str):
        self.marque = marque
        self.couleur = couleur

    def open_door(self):
        return "Door opened"
```

We can create an instance of the class:

```
my_car = Voiture("Renault", "noire")
```

The object `my_car` is an **instance** of the class `Voiture`.

### 8.1 Accessing a Method

Consider:

```
method_open = my_car.open_door
```

Notice that there are **no parentheses** after `open_door`.

Therefore, the method is **not executed**.

Instead, Python retrieves the method itself and stores a reference to it inside the variable `method_open`.

Conceptually:

|                                                                          |
|--------------------------------------------------------------------------|
| <code>method_open</code> $\longrightarrow$ <code>my_car.open_door</code> |
|--------------------------------------------------------------------------|

Thus:

```
type(method_open)
```

returns:

```
<class 'method'>
```

The variable `method_open` therefore contains a **method object**, not the result produced by the method.

## 8.2 Calling a Method

To **execute** the method, parentheses must be added:

```
my_car.open_door()
```

or, since the method has been stored in another variable:

```
method_open()
```

The parentheses `()` represent the **function/method call**.

The execution can be represented as:

|                                                                                                               |
|---------------------------------------------------------------------------------------------------------------|
| $\text{method\_open} \xrightarrow{()} \text{execute method} \xrightarrow{\text{return}} \text{"Door opened"}$ |
|---------------------------------------------------------------------------------------------------------------|

Therefore:

```
print(method_open())
```

produces:

```
Door opened
```

## 8.3 Method vs. Return Value

This distinction explains the following result:

```
print(type(method_open), type(method_open()))
```

which produces:

```
<class 'method'> <class 'str'>
```

Indeed:

|                                                           |        |                                                                              |
|-----------------------------------------------------------|--------|------------------------------------------------------------------------------|
| $\underbrace{\text{method\_open}}_{\text{method itself}}$ | $\neq$ | $\underbrace{\text{method\_open()}}_{\text{result of executing the method}}$ |
|-----------------------------------------------------------|--------|------------------------------------------------------------------------------|

More precisely:

$$\text{method\_open} \Rightarrow \text{method}$$

whereas:

$$\text{method\_open()} \Rightarrow \text{"Door opened"} \Rightarrow \text{str.}$$

The type of the result depends on what the method returns.

In this example:

```
def open_door(self):  
    return "Door opened"
```

returns a string, so:

```
type(method_open())
```

returns:

```
<class 'str'>
```

## 8.4 Storing the Method vs. Storing its Result

It is important to distinguish between:

```
method_open = my_car.open_door
```

and:

```
result = my_car.open_door()
```

In the first case:

```
method_open = the method itself
```

The method can therefore be executed later:

```
method_open()  
method_open()  
method_open()
```

Each instruction calls the method again.

In the second case:

```
result = the value returned by the method
```

Hence:

```
result = my_car.open_door()
```

```
print(result)
```

produces:

```
Door opened
```

Here, `result` is a string, not a method.

## 8.5 What Happens to `self`?

The method is defined as:

```
def open_door(self):  
    return "Door opened"
```

but we call it using:

```
my_car.open_door()
```

and not:

```
my_car.open_door(my_car)
```

This is because Python automatically passes the instance `my_car` as the first argument of the method.

Conceptually:

```
my_car.open_door()
```

is approximately equivalent to:

```
Voiture.open_door(my_car).
```

Thus, inside the method:

```
self = my_car
```

This allows the method to access the attributes of the specific instance, for example:

```
def description(self):  
    return f"{self.marque} - {self.couleur}"
```

## 8.6 Summary

| Expression                      | Meaning                          | Type                 |
|---------------------------------|----------------------------------|----------------------|
| <code>my_car</code>             | Instance of <code>Voiture</code> | <code>Voiture</code> |
| <code>my_car.open_door</code>   | Reference to the method          | <code>method</code>  |
| <code>my_car.open_door()</code> | Execution of the method          | <code>str</code>     |
| <code>method_open</code>        | Reference to the method          | <code>method</code>  |
| <code>method_open()</code>      | Execution of the stored method   | <code>str</code>     |

### Key idea:

`my_car.open_door` refers to the **method itself**, whereas `my_car.open_door()` **calls the method and returns its result**.

In Python, functions and methods are objects themselves. They can therefore be assigned to variables and called later.

## 9 Correction: Importing Data from an HTML Page

Consider the following code:

```
html = 'https://en.wikipedia.org/wiki/'
html = html + 'List_of_largest_oil_and_gas_companies_by_revenue'

stock_data = pd.read_html(html)

df = pd.DataFrame(data=stock_data[0])
```

### 9.1 What Does `pd.read_html()` Return?

The function

```
pd.read_html(html)
```

searches the HTML page for **HTML tables** and converts them into Pandas DataFrames.

An important point is that `read_html()` does not return a single DataFrame. It returns a **list of DataFrames**:

`pd.read_html(url) → [DataFrame0, DataFrame1, ..., DataFramen]`

Each element of the list corresponds to one table detected on the web page.

We can therefore inspect the number of tables using:

```
print(len(stock_data))
```

## 9.2 Selecting the First Table

The expression

```
stock_data[0]
```

selects the **first table** found on the page.

Since this object is already a DataFrame, writing

```
pd.DataFrame(data=stock_data[0])
```

is actually redundant.

We can simply write:

```
df = stock_data[0]
```

## 9.3 What Does the Extracted Data Look Like?

The extracted table contains information about large oil and gas companies and their revenues across different years.

Conceptually, the DataFrame has a structure similar to:

| Company   | 2019  | 2020  | 2021  | 2022  |
|-----------|-------|-------|-------|-------|
| Company A | 250.1 | 180.2 | 290.4 | 410.3 |
| Company B | 190.3 | 140.5 | 260.1 | 380.7 |
| Company C | 150.8 | 100.2 | 210.4 | 300.1 |
| ⋮         | ⋮     | ⋮     | ⋮     | ⋮     |

Each:

- **row** represents a company;
- **column** represents a variable or year;
- **cell** contains the corresponding value.

## 9.4 What Should We Do First?

When importing external data, we should **never immediately assume that the dataset is ready for analysis**.

The first step is to inspect its structure.

A useful initial diagnostic is:

```
df.head()
```

which displays the first five rows.

We should then examine:

```
df.shape
df.columns
df.index
df.dtypes
df.info()
```

These commands answer different questions:

| Command                 | Purpose                              |
|-------------------------|--------------------------------------|
| <code>df.head()</code>  | Inspect the first observations       |
| <code>df.shape</code>   | Number of rows and columns           |
| <code>df.columns</code> | Inspect column names                 |
| <code>df.index</code>   | Inspect the row index                |
| <code>df.dtypes</code>  | Check the data type of each column   |
| <code>df.info()</code>  | General structure and missing values |

## 9.5 What Problems Should We Look For?

Data extracted from an HTML page is **raw external data**. Before using it for quantitative analysis, several potential problems should be checked.

### 1. Missing Values

We can inspect missing observations using:

```
df.isna().sum()
```

Missing observations may require deletion, imputation, or another treatment depending on the application.

### 2. Incorrect Data Types

A column containing numbers may have been imported as strings:

```
df.dtypes
```

For example, values containing symbols, footnotes, or additional text may prevent Pandas from recognizing a column as numeric.

A numerical conversion may therefore be necessary:

```
df["2022"] = pd.to_numeric(df["2022"], errors="coerce")
```

### 3. Unnecessary Columns

HTML tables may contain columns that are not useful for our analysis.

They can be removed using:

```
df = df.drop(columns=["Column"])
```

### 4. Column Names

Column names may need to be simplified or standardized:

```
df.columns
```

For example:

```
df = df.rename(columns={"Company name": "company"})
```

### 5. Duplicates

Duplicated observations can be detected using:

```
df.duplicated().sum()
```

and removed, if appropriate, using:

```
df = df.drop_duplicates()
```

## 9.6 From Raw HTML Data to Usable Data

The general workflow is therefore:

HTML page → `read_html()` → Raw DataFrame → Inspection → Cleaning → Analysis

In practice:

```
# 1. Import
tables = pd.read_html(html)

# 2. Select the relevant table
df = tables[0]

# 3. Inspect
print(df.head())
print(df.shape)
print(df.dtypes)
```

```
print(df.isna().sum())
```

```
# 4. Clean
```

```
# ...
```

```
# 5. Analyze
```

```
# ...
```

**Key idea:** Importing data is only the first step. External data should always be **inspected, validated, and cleaned** before being used for statistical or financial analysis.